

AxiSim Software Manual v1.01

Lui Tsumura

May 2, 2026

Contents

1	Introduction	1
2	Interactive Functionalities	1
2.1	Mouse Picker	1
2.1.1	Collision with Bounding Box	2
2.2	Data Picker	2
2.2.1	Finding The Ray Cylinder Intersection	2
2.2.2	Bi-Linear Interpolation	3
3	Camera	4
3.1	Rotation	4
3.2	Pan	4
3.3	Axis Snapping	5
4	Mesh Generation	5
4.1	Control Volume	5
4.2	Data Structure and Index Notations	5

1 Introduction

This document is a software manual for the Axi-symmetric CFD solver using finite volume method, tentatively named AxiSim. Details regarding the methodology, design decisions, and software updates will be written primarily for documenting purposes.

Several new features were implemented and overhauled from version 1.0, as outlined in the progress report. The most notable features include the addition of the inspector GUI, the colormap GUI, an overhaul to the results and solver GUI, and save/load feature. The addition of texture buffers has also improved the performance of the display.

2 Interactive Functionalities

2.1 Mouse Picker

Mouse pickers are essential in 3D environments for selecting specific objects and enabling user interactions. This section describes how the mouse picker is implemented in AxiSim and their use cases. AxiSim uses ray casting to determine which object the user selected, so when the ray intersects an object's bounding box, the program executes the script associated with that object. The bounding box is defined by two opposing vertices and a unique ID. The unique ID is used to identify which bounding box was selected. Each vertex on the bounding box uses the `VertexLine` struct, where only the position and color is stored in a buffer.

2.1.1 Collision with Bounding Box

Collision between the ray and the bounding box is calculated using the slab method on the bounding boxes.

2.2 Data Picker

2.2.1 Finding The Ray Cylinder Intersection

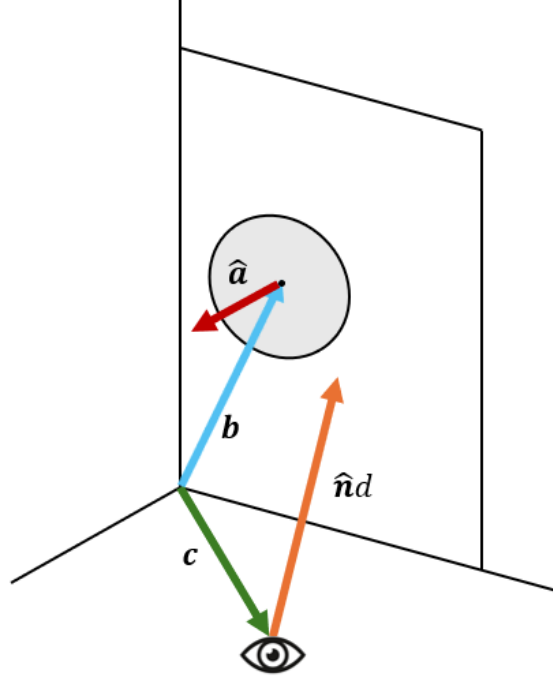


Figure 1: The ray hitting the plane but missing the end cap. Note the plane extends infinitely.

AxiSim lets users click on the surface of the mesh to obtain its corresponding value at that location. The first step in this process is finding the point of intersection between the ray and mesh, which is found using a line-cylinder intersection calculation. This calculation is done by first separating the mesh into two regions: the end caps and the outer surface. We first check if the ray intersects the end cap

$$(\hat{\mathbf{n}}d + \mathbf{c} - \mathbf{b}) \cdot \hat{\mathbf{a}} = 0$$

where $\hat{\mathbf{n}}$ is the ray unit vector, $\mathbf{c}$ the camera position, $\mathbf{b}$ the position of the end cap, and $\hat{\mathbf{a}}$ the unit normal vectors of the end cap. d is the distance until the ray hits the plane, which is found by rearranging the above equation.

$$d = \frac{\hat{\mathbf{a}} \cdot (\mathbf{b} - \mathbf{c})}{\hat{\mathbf{a}} \cdot \hat{\mathbf{n}}}$$

This gives the intersection point between the ray and the plane, which lies within the actual end cap if and only if

$$(\hat{\mathbf{n}}d + \mathbf{c} - \mathbf{b}) \cdot (\hat{\mathbf{n}}d + \mathbf{c} - \mathbf{b}) < r^2$$

where r is the radius of the end cap. This process is performed for both end caps, followed by determining the intersection point between the ray and the outer surface of the cylinder. This is determined by first computing the intersection between the ray and the cylinder's circular cross section. Unless the ray is

tangential to the circle, it produces two intersection points and therefore two corresponding distances: t_1 and t_2 . If an intersection exists, the program then checks whether the intersection point lies within the cylinder's length. If the intersection is within the cylinder's length, d is updated accordingly.

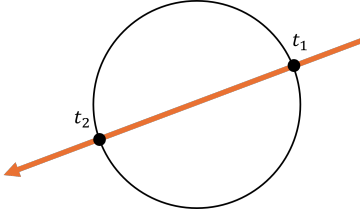


Figure 2: The ray intersecting with a cylinder.

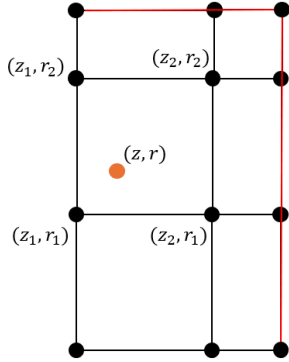


Figure 3: Demonstration of how the bilinear interpolation is implemented. At the boundaries the squares are smaller

In cases where multiple intersects are detected (ex: ray hits both the end caps and the outer surface), the smallest value of d is extracted. If the ray misses for any of the above case, d is not updated for that surface.

2.2.2 Bi-Linear Interpolation

Once the intersection point is determined for the ray and cylinder, the associated value at that location is determined using bi-linear interpolation; this is implemented in the `Results::getData` function. Because the solution is axisymmetric, the values are uniform in the angular direction and thus, only the axial and radial components are necessary to perform the interpolation. From the point of intersection, 4 data points on the bottom left, top left, bottom right, and top right are found using the 2D solution. Recall the concentration field has its values stored at the **cell center**, and therefore care must be taken when performing bi-linear interpolation near or at any of the boundaries.

3 Camera

The camera is implemented to allow the user to view and navigate the 3D space with ease. Care was taken when implementing the camera as it plays a major role in how the users view the virtual environment and their simulation results. This section provides a detailed explanation for the implementation, functionalities, and the decisions made to create a user friendly camera. In AxiSim, an arc ball camera is implemented using quaternions to simulate the CAD like style of control. The use of quaternions eliminates gimbal lock and simplifies rotational transformations. Every frame, the model, view, and perspective matrices are calculated and used to control the camera in 3D space. The model matrix is generally set to a 4x4 identity matrix unless a transformation is applied directly to the model. The view and perspective matrices are built using `glm::lookAt` and `glm::perspective` function. The position of the camera changes when there is rotation or translation applied by the user.

3.1 Rotation

When the user holds down the scroll wheel and drags the mouse, the camera moves along a spherical path around the target. The direction and angle of rotation is calculated using the mouse position, which is given in 2D coordinates bounded by the screen. These coordinates must be correctly mapped onto a 3D sphere in world coordinates so the camera can move accordingly. Once the coordinates on the sphere are obtained, the axis of rotation (normalized) and the degree of rotation must be determined using the previous cursor position $\hat{\mathbf{V}}_{\mathbf{a}}$ and current cursor position $\hat{\mathbf{V}}_{\mathbf{b}}$.

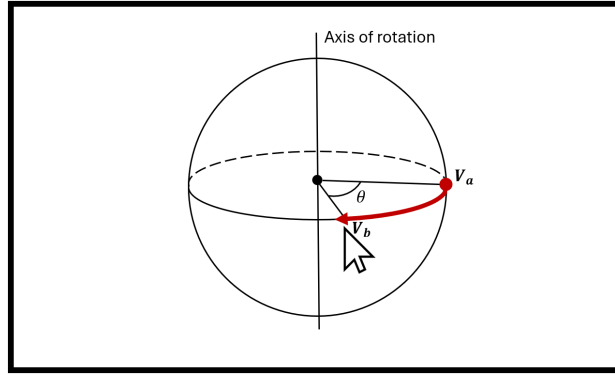


Figure 4: Visual representation of the arcball rotation

$$Axis = \frac{\hat{\mathbf{V}}_{\mathbf{a}} \times \hat{\mathbf{V}}_{\mathbf{b}}}{|\hat{\mathbf{V}}_{\mathbf{a}} \times \hat{\mathbf{V}}_{\mathbf{b}}|}$$

$$\theta = \arccos(\hat{\mathbf{V}}_{\mathbf{a}} \cdot \hat{\mathbf{V}}_{\mathbf{b}})$$

The quaternion is built using the `glm::angleAxis` function, and is used to update the rotation matrix for every frame a dragging motion is detected.

3.2 Pan

When the user clicks and drags the mouse the camera pans around the environment. As of v1.0, panning is handled by changing the target vector. The camera position is calculated as target + some offset, therefore changing the target vector also changes the camera position. This method is not the most user-friendly method as panning the camera also changes the point of rotation (center of arcball), which is very important for the user to know when rotating.

3.3 Axis Snapping

When the user clicks on one of the axis, the camera will automatically snap to that axis. Once the selected axis is identified from its corresponding ID, the script calculates the rotation matrix required for the camera to reach its final destination. This is accomplished by aligning the vectors defined by the camera and the selected axis. More specifically, the camera's front vector is aligned with the selected axis, and the camera's up vector is aligned with one of 6 basis vector, one for each axis. The former alignment moves the camera to the desired location and the latter alignment orients the camera properly. The following describes the steps taken to calculate the final target rotation quaternion.

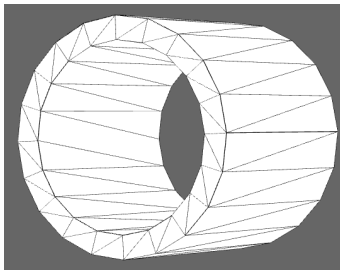
1. Compute quaternion needed to rotate the camera direction to the target axis
2. Using this rotation, compute the final camera up direction and compare it with a set of 6 candidate up vectors.
3. The candidate that best matches the final up direction is selected
4. The right, up, and forward vectors are used to construct the final target rotation for the camera.

Once the final target rotation is obtained, the script will move the camera every frame by performing an interpolation between the current quaternion and the target quaternion using the `glm::slerp` function. The interpolation factor is computed by finding the angle between the current and target quaternion. As the camera approaches the target orientation, this angle decreases, causing the interpolation factor to decrease accordingly, producing a smooth, nearly linear motion.

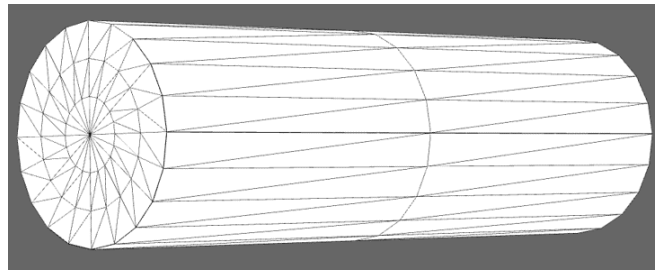
4 Mesh Generation

4.1 Control Volume

The mesh for an axi-symmetric finite volume method consists of several rings which are stacked radially and axially to create a cylinder. The generation process requires user inputs for the number of control volumes in the radial and axial directions (nr and nz), as well as the length and radius of the cylinder (L and R), and $nseg$, which is the number of segments used to divide the circle; larger values produce a smoother appearance. The default values for these variables are given in the Appendix. Each control volume consists of four faces (5a): front, outer, inner, and back face. However, the inner face is absent for the control volumes located along the centerline and thus only the front, outer, and back faces are considered.



(a) Sample control volume



(b) Example mesh for $nr = 3$ and $nz = 2$

Figure 5: Examples of the mesh and control-volume structure.

4.2 Data Structure and Index Notations

As there can be several thousand, if not millions of vertices that make up the mesh, it is important to organize them for ease of access and post processing. Three types of buffers are constructed to store and organize the vertices, namely the Vertex Attribute Object (VAO), Vertex Buffer Object (VBO) and the Element Buffer Object (EBO). In simple terms, the VBO stores the vertex data, the EBO defines how those vertices

are connected, and the VAO records how OpenGL reads and interprets the data. It is firstly important to understand how a vertex is defined in code. Each vertex has information regarding its **position**, and is stored in a VBO. The color of a vertex is calculated using its value, whether it be a value from velocity, pressure, etc at the given position. The normal vector points outwards from the surface and is used to compute the lighting, as described later. For each control volume, the vertices are identified and stored face by face in the following order: front, outer, inner, back.

	1st Row	Other rows
Front	$1 + nseg$	$nseg$
Outer	$2nseg$	$2nseg$
Inner	—	$2nseg$
Back	$1 + nseg$	$nseg$
Total	$nz(4nseg + 2)$	$6nseg$

Table 1: number of vertices for each face

The number of vertices for each face depends on the user defined value. This provides a general estimate of the memory required to store the mesh; for reference, generating a mesh with values $nseg = 20, nr = 170, nz = 100$ will yield about 2 million vertices. In addition to storing the vertices in a VBO, it is necessary to create an EBO to define how they are connected. OpenGL primarily works with triangles to render surfaces, meaning that each triangular surface consists of three vertices. We now define the **winding order**, which refers to the order in which the vertices are specified. OpenGL defaults to using counter-clockwise order, and this convention is adopted in this paper. Similar to how the vertices were stored, the indices are stored face by face in the same order. This way, each control volume can be reference easily as the indices are stored contiguously. However, the number of stored indices is significantly greater than the number of stored vertices, since every three indices define a single triangle. For example, in Fig 6 there are four vertices that define the quad, but six indices are required to draw the corresponding triangles. To store indices that make up each control volume, the ControlVolume struct is used, which holds information regarding each control volume's **size** (total number of indices), **starting index for each face** (front, outer, inner, and back), and the **number of indices** for each of those face.

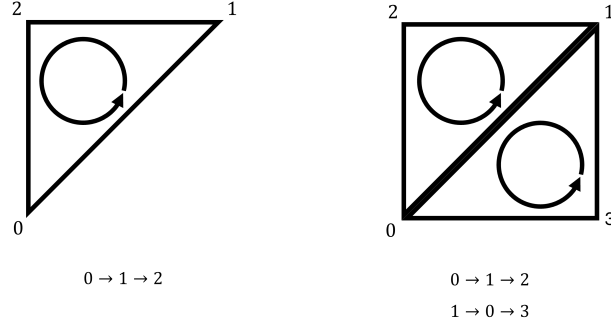


Figure 6: counter-clockwise winding order for a single triangle and a quad